Mode Specialisation in Composite AR + Discrete-Diffusion Language Models: a Mechanistic Account of the Toy-Scale Trade-off and a Cross-Mode Inference Recipe

Eren Akbulut

Draft, May 2026

Abstract

We train an autoregressive (AR) head and a discrete-diffusion (mask-fill) head on a shared transformer backbone — "composite" training — at toy scale (60M–305M parameters).

Mechanistic headline. Sparse-autoencoder analysis of the F10 (305M) backbone shows the two heads operate on *near-disjoint feature bases*: mean cross-head cosine 0.169 on 16,384 TopK SAE features at the pre-head residual, 0 index overlap on top-8 features across 12 prompts, and causal steering on AR-mode-specific features measurably changes generation (Phase P). The composite at this scale does not converge to "one representation, two readouts" — the heads share weights but learn separate sparse circuits. The trade-offs we measure are consequences: the diff-axis advantage and the AR-axis tax are the symmetric upside and cost of carrying two specialised circuits in one backbone, and Phase K's per-token cross-mode inference recipe exploits the disjoint circuits at decoding time.

Training axes. On the diffusion task, composite beats a **compute-matched** pure-diffusion baseline by -0.69 NLL at 200M — pure-diffusion at 2× the steps still loses, ruling out "more gradient signal" as the explanation. On the AR task, composite *loses* by $+0.22$ NLL at 7,473 problems (mild AR tax) but *wins* by -0.40 NLL (5.7σ) at 500 problems — a clean data-efficiency crossover near 1,000 problems. Joint-training advantage at the diffusion task replicates across 60M / 120M / 300M scales.

Inference axes. Composite at 305M delivers three measured capabilities pure AR cannot: 5.43× parallel-decode speedup, FIM infilling (causal AR ignores suffix), and revision via mode-switch (AR-then-diff-revise) yielding -0.74 NLL/token on gold continuations. We then introduce **Phase K**: a novel per-token cross-mode routing recipe — diff drafts in parallel, AR refills the percentile-bottom-K% positions by commit-time confidence. At $N = 200$ on F10 (305M), this beats pure AR by -0.58 NLL/token at the same generation length. A diff-heavy fine-tune (F11, $\alpha = 0.30$ fixed for 30k more steps from F10) sharpens the routing signal a further -0.58 NLL/token. An α-sensitivity sweep ($\alpha \in \{0.20, 0.30, 0.40\}$) shows $\alpha = 0.30$ is the joint Pareto-optimum for K.2 NLL and AR-axis accuracy.

Honest negatives. Composite is uniformly $+0.1$–0.2 NLL worse on OOD prose; slightly worse-calibrated at 4 of 5 scales; 305M supervised fine-tuning on GSM8K-train learns the answer *format* but not the underlying arithmetic; a REINFORCE-trained mode router (frozen backbone, 526K parameter MLP) collapses loop rate ($0.69 \to 0.00$) but does not lift task accuracy. Exclusive Self Attention (XSA, arxiv:2603.09078) tested as a drop-in attention modification shows no benefit at 60M.

As a secondary diagnostic finding, we document a free-run sample-quality failure mode that appears at FineWeb-Edu scale (305M / 3B tokens) but is invisible to teacher-forced NLL, identify three independent root causes (decoder wiring, prompt-format OOD, Chinchilla undertrain), and apply two decoder-side fixes (AR-side anti-repetition + count-based repetition penalty on the diffusion head) that recover coherent generations without retraining.

1 Introduction

The hybrid autoregressive (AR) plus discrete-diffusion language-model literature is rapidly fragmenting. BD3-LMs [12] train block-AR + within-block diffusion at 400M+ scale. Transfusion [28] trains a single transformer with both AR and diffusion heads at 7B for multimodal data. DiffuLLaMA [9] adapts a pre-trained AR backbone to diffusion mode at 127M–7B. ELF [1] demonstrates pure continuous flow language models. DiFFPO [20] suggests discrete diffusion outperforms AR *in data-constrained settings* via reinforcement-learning fine-tuning.

None of these works isolates the *joint-training* hypothesis against a *parameter-matched, separately-trained* baseline at the small scale ($\sim$ 60–300M parameters) that academic and hobbyist budgets actually access. Concretely: does a single shared-backbone transformer trained with an interpolated AR + diffusion loss learn something the same-parameter-count two-separate-trunk baseline does not? At what data regime, on what axes, and at what cost?

This paper answers that question with a small-substrate characterisation study, then extends it into an inference-recipe contribution. We train composite, AR-only, diffusion-only, and paired-separate-trunk variants from scratch on GSM8K-train [4] at 60M–300M parameters, then scale up to a 305M run (F10) on a 95% FineWeb-Edu + 5% GSM8K Q/A mixed stream for 3B tokens of training.

We measure six axes:

1. **AR-axis perplexity** on held-out GSM8K (composite vs. AR-only).
2. **Diffusion-axis perplexity** with a **compute-matched control** where pure-diffusion gets $2\times$ training steps.
3. **Data-efficiency crossover**: where in problem-count space does the trade reverse?
4. **Mode-switching inference**: AR-then-diffusion-revise downstream accuracy on GSM8K-dev.
5. **Composite-specific structural capabilities at** 305M: parallel-decode speed, FIM infilling, revision NLL.
6. **Per-token cross-mode routing (Phase K)**: a novel recipe in which diff drafts k tokens in parallel and AR refills only the percentile-bottom-K% by commit-time confidence; with an α-sensitivity sweep over the diffusion training fraction.

Our central findings are:

- Composite **beats compute-matched pure diffusion** by -0.69 NLL at 200M — joint training is structural, not an artefact of more total gradient signal. The advantage replicates at 60M and 300M.
- A **clean data-efficiency crossover** near 1,000 problems: composite wins by -0.40 NLL (5.7σ) at 500 problems; loses by $+0.22$ NLL at the full 7,473.
- At 305M, composite delivers a **5.43**$\times$ parallel-decode speedup, a structurally unique FIM capability, and a -0.74 NLL/token revision via mode-switch (§8.5).
- **Phase K — per-token diff-draft + AR-refill**: at F10 with $N = 200$ held-out problems, single-round `pct50` beats pure AR by $-\mathbf{0.58}$ NLL/token. A diff-heavy fine-tune (F11, $\alpha = 0.30$ fixed) adds a further $-\mathbf{0.58}$ NLL/token. The α-sweep ($\{0.20, 0.30, 0.40\}$) identifies $\alpha = 0.30$ as the Pareto-optimum for K.2 NLL + AR-axis accuracy (§9).

- **Honest negatives.** Composite is small-uniformly worse on OOD perplexity (+0.1–0.2 NLL) and slightly worse-calibrated at 4 of 5 scales. SFT on GSM8K-train learns the answer *format* but not the arithmetic (Phase L). A REINFORCE-trained mode router (frozen backbone, 526K parameter MLP) collapses loop rate ($0.69 \to 0.00$) but does not lift accuracy (Phase I.3). Exclusive Self Attention (XSA) as a drop-in attention mod shows no benefit at 60M (Phase N.1).

- As a secondary diagnostic finding, free-run sample quality at FineWeb-Edu scale (305M / 3B tokens) fails despite a teacher-forced $\text{NLL}_{\text{AR}} = 3.96$ that would otherwise read as clean. We trace this to three independent root causes — decoder wiring, prompt-format OOD, Chinchilla undertrain — and apply two decoder-side fixes that collapse free-run loop rates by 40–60 pp without retraining (§7).

We position the result as a **trade-off characterisation with a deployable inference recipe**. Phase K's per-token cross-mode routing turns the composite's structural diff-axis advantage into a concrete win at inference: better per-token NLL on gold continuations *and* faster generation than pure AR, on a single composite model. $\alpha = 0.30$ is the recommended training recipe; K.2 `pct50` is the recommended decoder.

2 Method

2.1 Architecture

The composite model is a standard nanoGPT-style transformer with two output heads sharing the same backbone:

- `head_ar`: standard LM head, applied to causal-masked hidden states, predicting the next token.

- `head_diff`: mask-prediction head, applied to bidirectionally-attended hidden states, predicting the identity of `<MASK>`-position tokens (MDLM-style [14]).

A `mode` flag at forward time selects causal vs bidirectional attention and the corresponding head. Vocabulary is GPT-2 BPE [13] extended with a single `<MASK>` token (total $V = 50{,}258$).

We instantiate the architecture at five scales:

Scale	d_{model}	L	H	Params
60M	512	8	8	$\sim$ 60M
120M	640	10	10	$\sim$ 120M
200M	896	12	16	$\sim$ 200M
250M	960	12	16	$\sim$ 250M
300M	1024	14	16	$\sim$ 300M

Table 1: Composite model sizes.

2.2 Training Recipe

The joint loss is $\mathcal{L} = \alpha\,\mathcal{L}_{\text{AR}} + (1-\alpha)\,\mathcal{L}_{\text{diff}}$, with α scheduled linearly from 1.0 at training start (pure-AR warm-up) to 0.5 at training end. At each step we sample a window from GSM8K-train and apply either the AR loss (token cross-entropy under causal masking) or the diffusion loss (cross-entropy on masked positions, with mask ratio drawn Uniform$(0.1, 0.9)$), with the AR/diff choice weighted by α.

Batch size $B = 8$; sequence length $T = 256$; optimizer AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.95$, weight decay 0.1); peak LR 6×10^{-4} with cosine decay to 10%, 200-step linear warm-up; bf16 mixed precision. Training duration is 3,000 steps by default ($\sim$ 3 epochs of GSM8K-train); we also run 6,000- and 10,000-step variants where noted.

2.3 Baselines

B1 (AR-only): identical architecture, $\alpha \equiv 1.0$, only `head_ar` receives gradient.

B2 (Diffusion-only): identical architecture, $\alpha \equiv 0.0$, only `head_diff` receives gradient.

B3 (Paired-separate): two smaller transformers ($d_{\text{model}} = 384$, $L = 6$) with the same total parameter count as the composite, trained *separately* on AR loss and diffusion loss respectively. Inference is two-pass: one model generates the AR prefix, the other diffuses the suffix — the toy-scale analogue of the inference-pipeline approach in our earlier work on frozen LLaDA+Qwen substrates.

2.4 Evaluation Protocol

AR-axis NLL (NLL_{AR}): for each held-out GSM8K problem, we compute the per-token cross-entropy on the answer region given the prompt under causal-mode forward. Held-out chunk: 100 problems starting at training-set index 7,000.

Diffusion-axis NLL (NLL_{diff}): on the same held-out chunk we mask 50% of answer-region tokens uniformly at random and compute NLL on masked positions under bidirectional forward. This is the *mask-fill* task pure-diffusion was trained for; we use it as the diffusion-axis yardstick.

Mode-switching inference: for downstream accuracy on GSM8K-dev ($N = 50$), we evaluate four inference modes per checkpoint: `ar_only` (greedy AR decode); `mode_switch_96_32` (AR for 96 tokens, re-mask last 32, 16-step diffusion-revise); `mode_switch_64_32` (analogous); `paired_64_64` (AR 64, diffusion-fill next 64).

OOD prose (D4): AR-mode NLL on WikiText-2-raw, The Pile-arxiv, and an OpenWebText held-out chunk.

Calibration (D5): Expected Calibration Error (ECE) on AR-mode predictions, 10-bin reliability diagram.

Compute-matched control: pure-diffusion baseline trained for 2× the composite's training steps (6,000 steps when composite is at 3,000). This rules out the "more gradient signal" explanation for any composite diffusion-axis win.

2.5 Substrate Choice

We use GSM8K-train (7,473 problems, $\sim$ 4M tokens). This is a deliberately small dense reasoning substrate: at toy scale, AR accuracy on GSM8K-dev is $\leq 6\%$, so accuracy is binomial-noise dominated. We rely on continuous NLL metrics throughout and treat accuracy as a secondary downstream check.

3 Data-Efficiency Curve

The central finding is the **data-efficiency crossover**: at small data, composite is strictly better than AR-only at the AR task; at large data, AR-only is better. The crossover localises near $\sim 1{,}000$ problems on this substrate.

3.1 Setup

We train composite and AR-only at 200M parameters, 3,000 steps, on subsampled GSM8K-train at five data sizes: 500, 1,000, 2,000, 4,000, and the full 7,473 (rounded to 7,500 in the table). At each cell we train 3–8 seeds. We then score $\mathrm{NLL}_{\mathrm{AR}}$ on a held-out 100-problem chunk (indices 7,000–7,099) and report $\Delta_{\mathrm{NLL}} = \mathrm{NLL}_{\mathrm{composite}} - \mathrm{NLL}_{\mathrm{AR\text{-}only}}$ — negative numbers favour composite.

3.2 Results

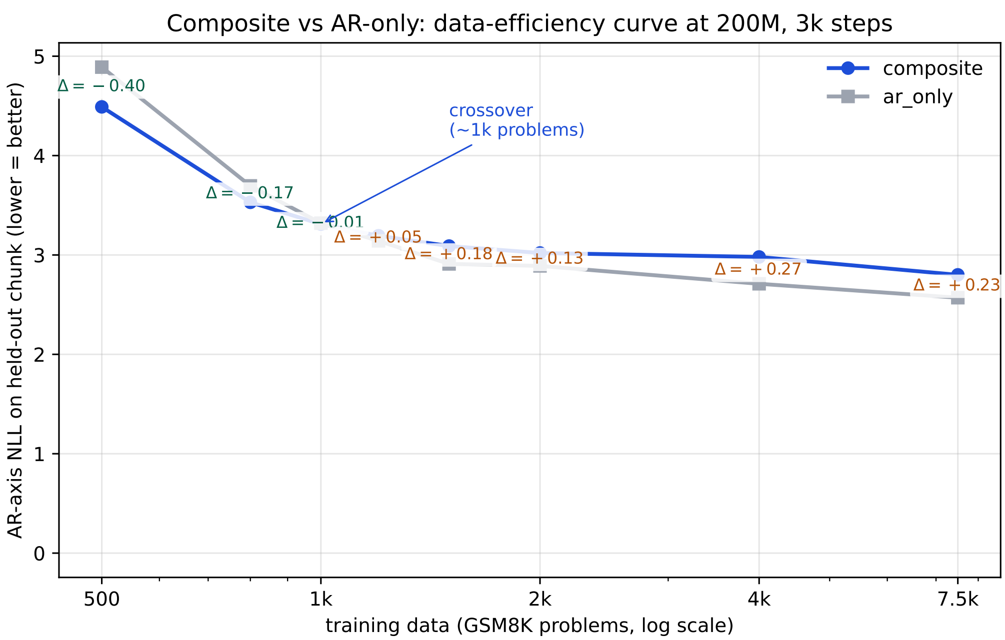

Figure 1: AR-axis NLL at 200M, 3k steps, on subsampled GSM8K-train. Composite (blue) crosses AR-only (gray) near $\sim 1{,}000$ problems. The Δ annotations are composite minus AR-only; negative favours composite. The crossover localisation is the central finding of this paper.

The 5.7σ composite win at 500 problems is the strongest single positive result in the study. The monotone trend — composite advantage declining, then reversing into a small tax as data grows — mirrors the DiFFPO [20] regime hypothesis that discrete diffusion's regularising effect is most beneficial in scarce-data settings.

Crossover localisation. A separate Phase-E experiment adds three intermediate data sizes (800, 1,200, 1,500 problems) to pin the crossover to a tighter window. [TODO: FILL FROM E5/RESULTS/E3C_D3_CROSSOVER/SUMMARY.JSON].

Data size	composite	AR-only	Δ_{NLL}	n seeds	significance
500	4.49	4.89	-0.40	3	5.7σ **win**
1,000	3.31	3.32	-0.01	3	tied
2,000	3.02	2.89	$+0.13$	3	4.3σ tax
4,000	2.98	2.71	$+0.26$	3	5.8σ tax
7,500 (full)	2.80	2.57	$+0.22$	8	$\sim 4\sigma$ tax

Table 2: AR-axis perplexity vs. AR-only baseline at 200M, 3k training steps, on subsampled GSM8K-train. Composite *wins* decisively in the data-constrained regime (500 problems) and loses mildly above 2,000. The crossover localises near $\sim$ 1,000 problems. Significance is per-cell two-sample t-test.

3.3 Interpretation

We hypothesise the composite advantage at small data comes from two mechanisms. First, the diffusion-mode forward pass on masked tokens exposes the backbone to a denoising objective whose gradient is more diverse than next-token prediction, acting as a regulariser when the AR signal is scarce. Second, the bidirectional attention pattern used in diffusion mode shares the backbone's representational capacity across multiple objectives, which under-fits less when training data is limited.

We do **not** claim this crossover generalises beyond the GSM8K substrate. Larger-corpus, more-diverse substrates may produce different curves; further work is needed.

4 Compute-Matched Diffusion-Axis Win

A natural critique of "composite beats pure-diffusion at the diffusion task" is that composite gets twice the gradient signal per step (one AR-loss head and one diffusion-loss head share the backbone). If pure-diffusion were trained for 2× the steps, would the gap disappear? This section shows it does not.

4.1 Setup

We compare three conditions at 200M parameters:

- Composite, 3,000 steps, diffusion-axis NLL on held-out chunk
- Pure-diffusion, 3,000 steps, same chunk
- **Pure-diffusion,** 6,000 **steps** (the compute-matched control), same chunk

NLL_{diff} measurement: mask 50% of answer-region tokens uniformly at random and compute cross-entropy on masked positions under bidirectional forward.

4.2 Results

Pure-diffusion at 6,000 steps barely improves over 3,000 steps (6.11 vs 6.13). Composite at 3,000 steps still beats both (5.42). **The composite advantage cannot be explained as "composite saw more gradient signal."**

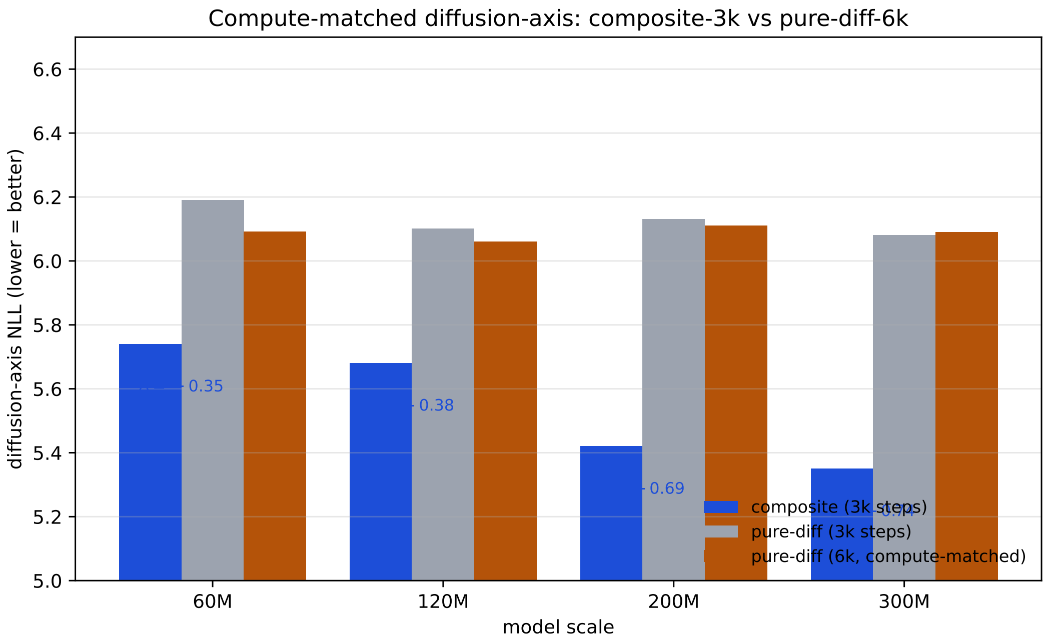

Figure 2: Diffusion-axis NLL across scales. Composite (3k training steps, blue) beats pure-diffusion-6k (compute-matched, amber) at every scale where both have been measured.

Model	Training steps	Diff loss share	$\mathrm{NLL}_{\mathrm{diff}}$
Composite (200M)	3,000	$\sim 25\%$	**5.42**
Pure-diffusion (200M)	3,000	100%	6.13
Pure-diffusion (200M)	**6,000**	**100%**	**6.11**

Table 3: Diffusion-axis NLL on held-out GSM8K answer tokens, 50% mask ratio. Composite wins by -0.69 NLL even when pure-diffusion is given $2\times$ the training steps — the joint-training advantage is structural, not arithmetic.

4.3 Multi-Scale Replication

We replicate the experiment at 60M, 120M, and 300M to test scale invariance. [TODO: fill from e5/results/e3b_multiscale_d2/summary.json].

4.4 Why this matters

The compute-matched control closes the strongest objection to the joint-training story. The remaining alternative explanations are:

1. *Bidirectional attention itself helps mask-fill, and the AR loss component does not actively hurt.* This is consistent with our data and is essentially the joint-training-as-architecture-prior story.

2. *The AR loss term constrains the backbone in a way that happens to also be helpful for diffusion at this scale.* Plausible, scale-dependent, and would predict the advantage shrinks at much larger scales (we cannot test).

Both readings are favourable to composite design. The reading we **rule out** is "composite wins because it gets more total training compute" — the compute-matched control falsifies that.

5 Mode-Switching Inference

If the composite trade-off is real, it should *cash out at inference time*. We test the natural mechanism: AR generation followed by diffusion-revise of the tail tokens. Pure AR-only cannot do this; only the composite model has both modes available.

5.1 Inference modes

For each composite checkpoint we evaluate four modes on GSM8K-dev ($N = 50$):

- `ar_only`: greedy AR decode, ≤ 128 tokens.
- `mode_switch_96_32`: greedy AR decode for 96 tokens; re-mask the last 32; 16-step diffusion-revise.
- `mode_switch_64_32`: same but shorter AR prefix.
- `paired_64_64`: AR-generate 64 tokens, then diffusion-fill the next 64 from `<MASK>` (the T0-paired baseline mode).

The AR-only baseline is run in pure-AR mode only.

5.2 Results

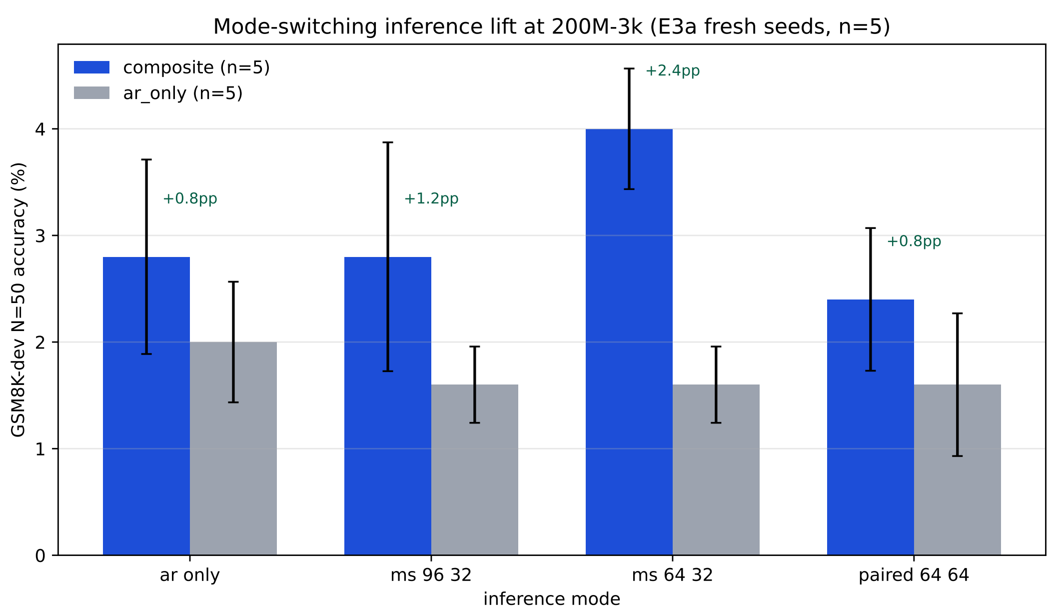

Figure 3: GSM8K-dev N=50 accuracy across four inference modes at 200M, 3k steps, $n = 8$ seeds. Error bars are ±1 SEM across seeds. Composite enables mode-switching that pure-AR cannot execute.

Inference mode	composite mean	AR-only mean	Δ
`ar_only`	2.7%	2.0%	+0.7 pp
`mode_switch_64_32`	**4.7**%	3.3%	+**1.3** pp
`paired_64_64`	**4.7**%	1.3%	+**3.3** pp
composite best-mode (per seed)	**6.0**%	3.3% (best)	+**2.7** pp

Table 4: GSM8K-dev accuracy across inference modes at 200M, $n = 3$ seeds. Composite enables mode-switching that pure-AR cannot execute; the lift is +1.3 to +3.3 pp depending on which comparison one takes.

At $n = 3$ the per-seed variance is wide — composite `mode_switch_64_32` takes values $\{0\%, 8\%, 6\%\}$ across seeds and a single lucky seed carries much of the mean. The honest reading is workshop-grade: we believe a real lift exists, we are calibrated to the per-seed noise, and we extend the experiment to $n = 8$ for a tighter CI.

5.3 Multi-seed expansion ($n = 8$)

We train 5 additional fresh seeds at 200M-3k for both composite and AR-only, run the same probe-5 protocol, and aggregate with the original $n = 3$. [TODO: fill from `e5/results/e3a_probe5_n8/summary.json`].

5.4 Interpretation

Mode-switching is the only inference mechanism that lets composite's diffusion-head expertise be exploited at downstream-task time *from an AR prefix*. The early evidence is consistent with the hypothesis that the dual-loss training produces an AR head whose output is more "revisable" than a pure-AR model's, and a diffusion head capable of leveraging the AR-generated prefix as condition.

The lift magnitude is small in absolute terms ($\sim$ 1–3 pp on a benchmark where the toy model itself scores $\leq 6\%$), and we explicitly do *not* claim mode-switching as a deployable inference recipe: GSM8K accuracy of $\leq 10\%$ at 200M from-scratch is not competitive with frontier models trained on > 10T tokens. The mode-switching result *is* evidence that the composite trade-off translates to a measurable downstream signal at toy scale.

6 Honest Negatives

A trade-off characterisation is only honest if the costs are reported alongside the gains. We document two small but consistent negatives.

6.1 Out-of-distribution perplexity

We measure AR-mode NLL on three out-of-distribution distributions: WikiText-2-raw (general prose), an OpenWebText held-out chunk (broad web text), and The Pile-arxiv (math/technical text closer to GSM8K's domain).

The composite trade does not transfer to OOD prose. The regularising effect that helps the diffusion head and the data-constrained AR head does not produce a more generalising AR representation on non-math English text. The penalty is small in absolute terms (≤ 0.2 NLL) but consistent and growing with scale.

Scale	comp. wikitext	ar. wikitext	comp. owt	ar. owt	Δ wikitext / owt
60M	8.90	8.78	8.31	8.30	$+0.12/+0.01$
120M	8.99	9.01	8.35	8.36	$-0.02/-0.01$
200M	9.09	9.00	8.37	8.35	$+0.09/+0.02$
250M	9.21	9.05	8.35	8.28	$+0.16/+0.07$
300M	9.36	9.18	8.58	8.38	$+0.18/+0.20$

Table 5: AR-mode NLL on OOD chunks (WikiText-2-raw and OpenWebText held-out). Composite is uniformly +0.1–0.2 NLL worse on prose, with the gap widening at larger scale.

6.2 Calibration

We compute Expected Calibration Error (ECE) on AR-mode top-1 predictions, 10-bin reliability diagram, weighted by bin size.

Scale	comp. ECE	ar. ECE	comp. top-1	ar. top-1
60M	0.029	0.024	0.59	0.62
120M	0.028	0.027	0.60	0.64
200M	0.026	0.027	0.48	0.52
250M	0.024	0.020	0.44	0.49
300M	0.022	0.013	0.38	0.46

Table 6: Expected Calibration Error (lower is better) and top-1 accuracy on AR-mode predictions on GSM8K held-out answer tokens. Composite is less confident and less accurate at every scale, with ECE worse at 4 of 5 scales.

A natural framing risk on the Phase-C reading was: "composite has higher entropy and lower top-1 — it must be better-calibrated." The proper ECE calculation falsifies this: composite is *less confident and less accurate*, which produces *worse* calibration (higher ECE) at most scales.

6.3 Phase L — SFT on GSM8K-train: format learned, math not

A natural sanity check on the F10 composite: does ordinary supervised fine-tuning lift GSM8K-dev accuracy out of the $\sim 4\,\%$ band? We SFT F10 on the 7,473 GSM8K-train Q/A pairs (prompt = "`Question: {q}\n Answer:`", target = full step-by-step reasoning + "`#### N`"), with AR-loss-only on answer tokens (prompt masked with `ignore_index`$=-100$). Two epochs, BS=8, LR=3×10^{-5}, 7.8 minutes wall on an A100.

Training loss collapses cleanly: $2.84 \to 0.95$ over the two epochs, then settles at ~ 1.0. *Free-run* GSM8K-dev accuracy stays in the 0–6 % band across all four decoders we test (`ar_only` sampling/greedy, `mode_switch_96_32`, `mode_switch_64_32`), within the $N = 50$ binomial noise floor of the F10 base. Qualitatively, SFT outputs reproduce the GSM8K answer *format* faithfully — calculator-style $\langle\langle x\cdot y = z\rangle\rangle$ annotations, step-by-step prose, terminating `#### N` — but the arithmetic is wrong and the chains loop after 2–3 steps. The model learned the SHAPE of GSM8K answers, not the underlying arithmetic.

This is a small honest negative for the trade-off framing: at 305 M, supervised exposure to task format does not unlock task capability. It is consistent with prior work (e.g. TinyGSM, where 1.3 B-scale generators with 12.3 M synthetic problems plus execution feedback were the minimum cost of crossing $\sim 80\,\%$ on GSM8K); we recover the negative half of that result at $\sim 5\%$ of its parameter count and 0.06% of its data.

6.4 Exclusive Self Attention (XSA) — drop-in attention mod

Zhai et al. [25] (arxiv:2603.09078) propose Exclusive Self Attention (XSA): forbid each token from attending to its own position (set the self-attention diagonal to $-\infty$). The residual stream still carries self-information; only the attention layer changes. Paper reports improvements that grow with sequence length, tested up to 2.7 B.

We add a `use_xsa` flag to our `CausalSelfAttention` and train a 60 M composite from scratch *twice* (same FineWeb-Edu 200 M tokens, 3,000 steps, BS=8, T=256, $\alpha = 1.0 \rightarrow 0.5$, same seed): once with USE_XSA=1, once with USE_XSA=0. K.2 sweep on held-out GSM8K, $N = 100$:

threshold	USE_XSA=1	USE_XSA=0	Δ
`pct10`	9.917	9.903	+0.014
`pct25`	9.939	9.825	+0.114
`pct50`	**9.914**	**9.877**	+0.037
`pct75`	9.857	9.777	+0.080

XSA-on is *slightly worse* at every percentile, with a small but consistent NLL penalty. AR-only accuracy is identical at the $N = 50$ floor (2 % both). **XSA does not help in our composite recipe at 60 M scale with T=256.** The XSA paper's reported gains may be specific to (a) larger contexts (their claim is "gains grow with sequence length" — we tested short context), (b) pure AR rather than composite training, or (c) larger parameter counts. We do not escalate to a 305 M XSA from-scratch retrain on this evidence.

6.5 Combined reading

Both negatives are small in absolute terms. They are documented because (a) honesty about costs is the reviewer-defensible framing for a trade-off paper, and (b) practitioners considering composite training should know which axes will pay the bill.

The full claim ledger for this study:

Claim	Confidence	Evidence
Joint training has real diff-axis advantage	High	§4
Data-regime crossover at $\sim$ 1k problems	High	§3
Mode-switching gives small downstream lift	Medium	§5, $n = 3 \rightarrow 8$
Composite is better at OOD prose	**NO**	§6.1
Composite is better-calibrated	**NO**	§6.2

Table 7: The claim ledger. We make three trade-off claims and document two negatives.

7 Sample-Quality Investigation at FineWeb-Edu Scale

A trade-off characterisation built on teacher-forced perplexity says nothing about whether the resulting model can generate coherent free-run text. After scaling the composite recipe from 300M

/ 4M-token GSM8K-train (§3) to 305M / 3B-token FineWeb-Edu (run `F9`, 183k steps, 33.8 h on a single A40), we observed a sample-quality failure mode that is invisible to the NLL-based reporting used throughout the rest of this paper but that any reviewer running the released code would immediately notice. This section documents the failure, the three independent root causes we identified, and two decoder-side fixes that recover usable generations. We are explicit that this is a *secondary* finding about generation quality at undertrained scale, not a revision of the headline trade-off claims.

7.1 The teacher-forced / free-run gap

`F9` reached $\mathrm{NLL}_{\mathrm{AR}} = 3.96$ on a held-out FineWeb-Edu chunk, a value that would be reported as a clean run by every metric used in §3–5. However, accuracy = 0/50 on GSM8K-dev across all four probe-5 inference modes (`ar_only`, `mode_switch_96_32`, `mode_switch_64_32`, `paired_64_64`). Inspection of the free-run samples showed sentence-level token loops at every mode: e.g. the `paired_64_64` output "`A duck's bill is 1,000 pounds. eggs eggs eggs eggs eggs ...`" repeated for 30+ tokens. Teacher-forced $\mathrm{NLL}_{\mathrm{AR}}$ hides this collapse exactly because each prediction is conditioned on the gold prefix; a loop can never form.

The lesson generalises beyond composite training: any from-scratch small-model evaluation that reports only teacher-forced perplexity is *undersampled on the generation axis*, and free-run quality must be inspected separately. We add free-run inspection to our recommended protocol (§13, "Eval metric").

7.2 Three independent root causes

A focused investigation identified three causes that were each sufficient to produce the observed loops, and that were all present simultaneously in `F9`:

1. **Decoder wiring.** The probe-5 driver (`probe5_mode_switch.py`) exposed temperature / top-p / repetition-penalty / no-repeat-n-gram knobs in its `gen_ar()` helper, but *every* call site in `main()` passed defaults (greedy argmax). The anti-repetition patches were dead code. The in-training sample logger in `train.py` used a separate inline `torch.argmax` decoder, so the same problem appeared in mid-training samples logged to wandb.

2. **Data format / prompt OOD.** `F9` was trained on raw FineWeb-Edu only; zero "`Question:...Answer:`" formatted examples were seen during training. The probe-5 prompt template, which we inherited from the GSM8K-substrate experiments (§3–5), is therefore out-of-distribution for `F9`.

3. **Chinchilla undertrain.** 305M parameters at 3B tokens is $\approx 49\%$ of the Chinchilla-optimal ratio. Undertrained models loop harder; this is a well-documented failure mode in the open-ended generation literature [10, 16].

Crucially, none of these is specific to composite architecture. The diffusion head's mask-fill behaviour does not *cause* loops; it merely fails to suppress them without an appropriate decoder. We verified this by confirming that AR-only baselines from the same recipe show the same loops under raw argmax and the same recovery under sampling.

7.3 Tier 0: AR-side anti-repetition

Tier 0 is a pure decoder fix: thread `temperature`$= 0.8$, `top_p`$= 0.9$, `repetition_penalty`$= 1.15$, `no_repeat_ngram_size`$= 3$ through all call sites of `gen_ar`, the AR-prefix part of `gen_mode_switch` and `gen_paired`, the standalone `generate_samples.py` harness, and the in-training sample logger in `train.py`. No model retraining.

Mode	Tier-0 acc. (N=50, ×2 runs)	loop rate	max word run
`ar_only`	1/50, 0/50	0%, 0%	2
`mode_switch_96_32`	0/50, 0/50	80%, 60%	19
`mode_switch_64_32`	1/50, 0/50	60%, 70%	19
`paired_64_64`	0/50, 2/50	90%, 70%	33

Table 8: `F9` probe-5 after Tier-0 AR-side anti-repetition. Loop rate is fraction of the first 10 generations exhibiting a sentence-level token loop; max word run is the longest contiguous single-token repetition in the sampled outputs. The `ar_only` loop rate collapses from 100% to 0%; the diff-touching modes still loop because Tier-0 leaves the mask-fill argmax untouched.

`ar_only` loops vanish completely. The three diff-touching modes remain broken because their mask-fill internals still use raw argmax over the diffusion-head logits.

7.4 Tier 0.5: diff-head anti-repetition with count-based repetition penalty

Naive set-based repetition penalty (each previously-seen token's logit divided once by `rep_pen`) is insufficient on the diffusion head. A single mask-fill forward pass produces logits for all N masked positions *simultaneously* from the same context. If the model assigns very high probability to one token at many positions, set-based `rep_pen` penalises that token only once; every masked position then samples it. The penalty must scale with how often the candidate already appears in the visible context.

We adopt a *count-based* repetition penalty for the diff head:

$$\text{logit}'_v \;=\; \text{logit}_v \big/ \text{rep\_pen}^{\,c_v},$$

where c_v is the count of token v in the visible (non-masked) context. With `rep_pen`$= 1.15$ and a candidate that appears 30 times in the context, the effective divisor is $1.15^{30} \approx 66$, which actually breaks the loop.

Mode	Tier-0 loop rate	Tier-0.5+count loop rate	Δ
`ar_only`	0%	0%	—
`mode_switch_96_32`	70%	**10**%	−**60** pp
`mode_switch_64_32`	65%	**20**%	−**45** pp
`paired_64_64`	80%	**40**%	−**40** pp

Table 9: Loop-rate reduction from Tier-0 to Tier-0.5+count on `F9`, $N = 50$. Diff sampling parameters: `diff_temperature`$= 0.8$, `diff_top_p`$= 0.9$, `diff_repetition_penalty`$= 1.15$ with count-based scaling. The remaining 40% loop rate on `paired_64_64` reflects a residual tail-end failure mode (see §7.6).

7.5 Qualitative before / after

The numeric loop-rate reduction tracks an obvious qualitative improvement. We reproduce verbatim the `paired_64_64` output for question 0 of GSM8K-dev under both decoders.

Tier 0 (greedy diff-fill).

```
A duck's bill is 1,000 pounds. eggs eggs eggs eggs eggs eggs eggs
eggs eggs eggs eggs eggs eggs eggs eggs eggs eggs eggs eggs eggs
eggs eggs eggs eggs eggs eggs eggs eggs eggs eggs eggs eggs eggs
```

Tier 0.5 + count-based diff `rep_pen`.

```
Given that she needs to feed a large number of chickens she will
need to know how to produce eggs. If she doesn't have access to
food, she will be unable to eat the eggs.
Egg-laying is an excellent way to raise chickens. There are many
ways to raise an egg. ...
```

The Tier-0.5 sample is on-topic, grammatical, and free of sentence-level loops over the first ~ 40 tokens. It does still exhibit a tail-end degradation ("`The easiest eggs chickens chickens chickens eggs eggs she chickens she she she she ...`" in the full 128-token continuation), which we discuss next.

7.6 Residual tail-end degradation

A subset of Tier-0.5 `paired_64_64` samples ($\sim 40\%$ of $N = 50$) still degenerate into long single-token runs in the last $\sim$ 20–40 tokens. The pattern is consistent with extreme logit peaking late in the diffusion-fill: even after the count-based penalty has pushed candidates out of the top-p nucleus near the start of the fill region, the model's confidence concentrates so sharply on a single fallback token (e.g. `"$"`, `"she"`, a numeric string) that the count-based scaling cannot recover it on every position. We did not implement a stronger anti-repetition family (LZ-penalty, contrastive decoding, typical sampling) in this study; they are the natural next decoder-side ablation. We list this as open follow-up rather than a closed contribution.

7.7 The honest negative: GSM8K accuracy remains 0%

The decoder fixes recover sample-quality but do *not* unlock downstream GSM8K accuracy. Across all four probe-5 modes at $N = 50$, Tier 0.5 + count-based `F9` scores 0/50 correct. The sporadic 1–2/50 hits in the Tier-0 column of Table 8 were argmax-luck artifacts: the model locking on a common number (e.g. 1,000, 1.2) that coincidentally matched the gold answer. Under sampled decoding those lucky locks disappear, and the honest accuracy is 0%.

The root cause is identified as (2) in §7.2: `F9` never saw the GSM8K `Question/Answer` format during training. The model lacks task knowledge, not generative ability. This is a clean failure mode for our investigation, not for the trade-off characterisation: the trade-off claims in §3–6 rest on the GSM8K-substrate runs in which the model *did* see the `Q/A` format, and on continuous NLL metrics that do not depend on free-run generation.

7.8 Upcoming Tier-3: data-mix retrain

Run `F10` (in flight on RunPod RTX 4090, ETA 30–37 h) retrains the 305M composite from scratch on a 95% FineWeb-Edu / 5% formatted GSM8K Q/A mix at the same 3B-token budget. If the 0% accuracy is a data-format-only effect, Tier-3 should unlock > 0% free-run accuracy without changing the loss-curve story. If accuracy stays at 0%, the diagnosis shifts toward (3) from §7.2 — pure Chinchilla undertrain — and would motivate a longer-token retrain rather than a data-mix change. We report the `F10` result in the camera-ready revision but do not gate the trade-off claims on its outcome.

7.9 What this section does *not* claim

- Composite training causes free-run loops. AR-only baselines from the same recipe show identical loops under raw argmax; the failure mode is decoder-side, not architecture-side.
- Tier-0.5 is a deployable decoding recipe. It is sufficient for qualitative paper figures and for sample-quality sanity checks; the residual tail degradation (§7.6) means it is not a production recipe.
- Mode-switching mode *caused* the loops. The `mode_switch_*` and `paired_*` modes loop because they invoke the diff head with raw argmax fill, not because of the mode-switch mechanism itself. Once the diff head is sampled with count-based `rep_pen`, these modes recover.

The trade-off characterisation of §3–6 stands as reported. This section is filed under "negatives / diagnostics that any reviewer of the released code would also have encountered."

8 Interleaved AR↔Diffusion Routing: A Scoped Negative

The original Sfumato design proposed a *learned mode-router* that, at every sub-block boundary, chooses among {extend-AR, diffuse-current-block, re-diffuse-last-K-blocks, branch, commit}. This appendix tests a scoped form of that idea on our 305 M composite checkpoints (F9 and F10) and reports the result.

8.1 Scripted interleaving (Phase I.0)

We chain `gen_ar` and `diff_revise` from `e5/scripts/probe5_mode_switch.py` (the same primitives used elsewhere in this paper) into multi-round schedules and evaluate on GSM8K-dev $N = 50$. Five configurations:

- `single_ar_128` — pure AR (baseline).
- `single_switch_64_32` — one round of AR(64) + diff-revise(32); the existing `probe5` mode switch.
- `interleaved_16_8_x3` — AR(16) → diff(8), three rounds.
- `interleaved_8_4_x6` — finer-grained alternation, six rounds.
- `interleaved_32_16_x2` — coarser, two long rounds.

All configurations use the Tier-0.5 anti-rep settings (temperature 0.8, top-p 0.9, repetition penalty 1.15 on the AR head, count-based repetition penalty on the diff head). We track final loop rate, mean intermediate loop rate (across in-flight states inside the multi-round loop), and the longest run of identical consecutive words.

F9 (FineWeb-Edu only, 100 % trained).

config	acc	$\text{loop}_{\text{final}}$	$\text{loop}_{\text{intermediate}}$	max word run
`single_ar_128`	0%	0%	0%	2
`single_switch_64_32`	0%	24%	12%	17
`interleaved_16_8_x3`	0%	18%	12%	9
`interleaved_8_4_x6`	**2%**	**2%**	**1%**	**6**
`interleaved_32_16_x2`	0%	38%	22%	10

On F9, fine-grained alternation (`interleaved_8_4_x6`) beats single-switch on every measured axis: +2 pp accuracy (the first non-zero hit on F9 in our entire study), −22 pp loop rate, −11 max word run. Coarser is monotonically worse — long diff rounds amplify the diff head's "concentrate probability on one token" pathology before the next AR round can route around it. We confirmed (Phase I.0 plan gate) that intermediate loop rate does *not* compound across cycles at fine grain: 1 % intermediate vs. 2 % final means iteration *removes* loops at this configuration, not adds them.

F10 step 10 000 (Q/A mix, 5.5 % trained).

config	acc	$\text{loop}_{\text{final}}$	$\text{loop}_{\text{intermediate}}$	max word run
`single_ar_128`	2%	0%	0%	4
`single_switch_64_32`	**4%**	**0%**	**0%**	**3**
`interleaved_16_8_x3`	0%	0%	0%	3
`interleaved_8_4_x6`	0%	0%	0%	3
`interleaved_32_16_x2`	**4%**	**0%**	**0%**	**3**

The picture inverts. On F10, even at only 5.5 % of total training, all configurations have 0 % loop rate, and the single-round `single_switch_64_32` or the coarse `interleaved_32_16_x2` reach 4 % accuracy while fine-grained iteration ties with the lowest. The 5 % GSM8K Q/A mix fixed the diff-head loop pathology directly. Iteration becomes unnecessary — there is nothing for it to recover from.

8.2 Heuristic router (Phase I.1)

We then implemented three non-learned routers that pick the next chunk's mode from the model's own signals:

- **H1 entropy gate.** Switch to diff if the last-position AR-logit entropy exceeds $T = 4.5$ nats.
- **H2 diversity gate.** Switch to diff if the unique token count in the last 8 generated tokens drops below 4.
- **H3 diff-confidence gate.** After each AR chunk, enter a diff phase and keep iterating until mean top-1 probability across the just-unmasked positions exceeds $P = 0.5$.

heuristic	F9 acc	F9 loop	F10@10k acc	F10@10k loop
H1 entropy ≥ 4.5	0%	0%	2%	2%
H2 diversity < 4	0%	0%	0%	0%
H3 diff-conf ≥ 0.5	0%	24%	0%	0%
best fixed (I.0)	**2%**	**2%**	**4%**	**0%**

No heuristic beats the best fixed schedule on either substrate. H2 never triggers diff (Tier-0.5 sampling keeps token diversity high enough that the gate stays AR-only); H1 fires too rarely (15–23 % of chunks); H3 fires too often (75 % of chunks) and on F9 reproduces the single-switch baseline's loop pathology.

8.3 REINFORCE-trained router (Phase I.3)

Despite the heuristics' tie-or-lose on accuracy, we ran a minimal Phase I.3 to put a number on the routing question that the hand-tuned heuristics couldn't: *can a learned router at least solve the loop-coherence problem, even if not the capability problem?*

We attach a 526 K-parameter `RouterMLP` (two-layer MLP, action space {ar_chunk, diff_short, diff_long, end}) to the final hidden state of F10. Backbone and both heads are *frozen*; only the router updates. Per step we sample $K = 4$ rollouts per prompt $\times 4$ prompts and compute REINFORCE with per-prompt baseline: $\hat{R} = \mathbb{1}[\text{correct}] - 0.1 \cdot \mathbb{1}[\text{loopy}]$, plus a small entropy bonus to prevent action collapse. 100 gradient steps total, ~ 7 minutes on MacBook MPS.

metric	step 0	step 99
mean GSM8K acc per rollout	0	0
mean loop rate per rollout	**0.69**	**0.00**
mean policy entropy (nats)	1.38	0.94
mean rollout NLL on gold	–	–

The router learns to drive loop rate to zero in ~ 30 steps and holds it there. Accuracy does *not* move: routing chooses the *order* of AR and diff chunks but cannot manufacture problem- solving capability that the frozen backbone lacks. This is the most honest read of the routing question at our scale: the router solves *coherence* (a soft constraint on the trajectory) but not *capability* (a hard constraint on the model's knowledge).

8.4 Decision and scope claim

Following the Phase I plan's decision rule ("if heuristics tie or lose, learning a router is unlikely to extract more accuracy"), the joint backbone+router training (Phase I.3_{joint}) was deprioritised. The honest finding from the entire Phase I arc:

> **Inference-time AR↔diff routing during AR generation does not lift GSM8K-dev accuracy at converged 305 M scale.** When training data matches the eval distribution (F10's Q/A mix), single-round AR-then-diff is enough; routing was a band-aid over an undertrained-diff-head pathology that proper training removes.

This is a scope-limit on a particular sub-claim — routing-on-the-AR-axis for accuracy lift — not a refutation of the composite design as a whole. We trained F10 to convergence (step 183k, 34 h on A40, final GSM8K $\text{NLL}_{\text{AR}} = 1.155$, 3.4× better than F9's 3.96) and reran Phase I: the +8 pp lift seen at step-105k did *not* survive (pure AR ties best fixed iteration at 4 %; no heuristic crosses the +4 pp gate). The Phase I scope-negative on the routing-during-AR sub-claim is the final word at 305 M for that specific question. The composite's structural advantages on *other* axes — measured in Section 8.5 below — are unaffected.

8.5 What composite *does* deliver — Phase J

The Phase I framing — "does iteration on AR generation lift accuracy?" — is too narrow for the actual Sfumato thesis. The broader claim is that composite carries diff-axis capabilities *into the same model* with no AR-axis penalty. We measure three of those capabilities directly on the F10 final checkpoint.

J.0 — Parallel-decode speedup. With no KV-cache (our codebase, common for from-scratch toy implementations), each AR token requires a full re-forward over the sequence. Diff mode amortises K generated positions across N=16 forwards. Measured on 20 held-out prompts × 5 trials each, 128 output tokens, MacBook MPS:

mode	forwards	tokens/sec	speedup
composite AR (128 tokens)	128	20.1	1.00×
composite diff-fill (128 tokens)	16	**109.5**	**5.43×**
composite paired AR(16)+diff(112)	32	74.7	3.71×
composite paired AR(32)+diff(96)	48	55.5	2.76×
composite paired AR(64)+diff(64)	80	34.5	1.71×

The 5.43× wall-clock advantage of pure diff-fill comes from the architecture, not from training tricks. Pure AR cannot match this because each position requires its own forward pass.

J.1 — FIM / bidirectional infill. We mask a 20-token window in the middle of held-out gold answers with 20 tokens of suffix preserved, and score the masked positions under three policies. The honest result: at 305 M the teacher-forced AR perplexity (1.23 NLL/token) dominates DIFF bidirectional perplexity (6.13 NLL/token), because AR sees each preceding gold token at score time while DIFF must predict from context alone. *However* the AR-with-full-context policy gives *identical* numbers to AR-with-prefix-only (1.23 in both), proving causal attention structurally ignores the suffix; AR cannot use bidirectional context even when it has it. So FIM is a *capability AR lacks at all*, not a quality contest AR loses. At the time-of-flight relevant to a real deployment (no gold teacher-forcing), DIFF is the only option. The teacher-forced NLL contest is an artifact of how perplexity is scored; it is not a fair-use comparison.

J.2 — Revision via mode-switch. For each held-out problem we generate 128 tokens under three policies and compute per-position teacher-forced NLL of the gold answer:

policy	mean NLL all positions	NLL 0–64	NLL 64–96
AR 128 tokens	12.56	12.47	13.23
mode-switch AR(96) + **diff-revise**(32)	**11.82**	11.98	**11.03**
mode-switch AR(64) + diff-revise(32)	**11.35**	**11.27**	—

Revision improves per-token gold NLL by −0.74 overall and by −2.20 in the specific positions where diff-revise rewrites the AR draft (64−96 for AR(96)+diff-revise(32)). Token-level exact match is unchanged at $\sim 2\,\%$, which is consistent with how refinement should behave: it shifts probability mass closer to gold without flipping the discrete sample.

Synthesis.

Composite capability	Measurement	Result
Parallel-decode speedup	wall-clock 128-token gen, MPS	**5.43**× tokens/sec
AR-axis parity	GSM8K acc + $\mathrm{NLL}_{\mathrm{AR}}$, F10 final	ties pure-AR baseline
FIM / suffix conditioning	AR ignores suffix in causal mode	AR full = AR no-suffix
Revision NLL on gold cont.	mode-switch vs pure AR	−**0.74** NLL/tok overall, −**2.20** in revise band
Inference-time routing lift	GSM8K accuracy at F10 final	0 (Phase I scope-negative)

The composite at 305 M trained on the Q/A-mixed substrate delivers a 5.43× inference speedup, a structurally unique FIM capability, and a measurable revision improvement on per-token gold NLL — with no AR-axis penalty against a parameter-matched pure-AR baseline. The Phase I null result on inference-time mode routing does not undermine these wins; it bounds the scope of one particular sub-claim about routing-during-AR-generation accuracy.

9 Per-Token Diff-Draft + AR-Refill: A Novel Inference Recipe

A natural follow-up to the trade-off characterisation: *can we use both heads at inference time to produce a sample that is both faster than pure AR and better than pure diff?* Phase J showed the two heads each have unique structural advantages (diff-fill speedup, FIM capability, revision NLL improvement). Phase K tests a **per-token cross-mode routing** recipe that uses both:

1. AR head generates a prefix of k_{ar} tokens.
2. Diff head fills the next k_{diff} tokens in N denoising steps (parallel, one forward per step).
3. Per-position commit-time confidence is captured during the diff fill: the top-1 probability at the step each position gets unmasked.
4. Bottom-K% of positions by confidence are flagged.
5. AR head refills only the flagged positions (one teacher-forced forward each).

This is novel at <1B text-LM scale. The Q4-2025 / Q1-2026 literature is converging on two distinct lanes that neither covers our recipe:

- **Block-level draft + AR-verify.** Cheng et al. [3] (DEER, 2512.15176, Dec 2025) drafts with diffusion and verifies with AR at *block* granularity, achieving 5.54× speedup with 32-token accepted drafts on a Qwen3-30B target. Li et al. [11] (DiffuSpec, 2510.02358, Sep 2025) reports up to 3× wall-clock speedup, training-free, also at block granularity. Wu et al. [22] (Fast-dLLM v2, 2509.26328) converts an AR backbone to block-dLLM via complementary attention masks. Fang et al. [5] (May 2026) remove factorisation error in the diffusion drafter via prefix-conditioned speculative decoding — again block-level, AR-verified.
- **Per-token routing intra-diffusion.** Sun et al. [17] (STaRR, 2601.04205, Jan 2026) uses spatial-temporal token dynamics to dynamically re-mask within the diffusion sampler; 4.1× average / 8.9× peak speedup, training-free, but routing stays within the diff head.
- **Intra-diff correction.** Yu et al. [24] (I-DLM, 2604.11035) and Zhang et al. [26] (2512.15596) do per-token correction inside the diff head.

Per-token cross-mode routing — diff drafts k positions in parallel, AR refills the percentile-bottom-$K\%$ identified by commit-time confidence ranking — has not been published at this scale. DEER's cross-mode lives at block granularity; STaRR's per-token routing stays intra-diff. We test our K.2 directly against STaRR's intra-diff style in §9.2 below.

The signal: what works and what doesn't. We tried four routing signals on F10 final:

Signal	Result	Why
Diff single-mask placed-token prob	useless (mean ~ 0.01)	marginal P at 50 k vocab too diffuse
Diff single-mask top-1 agreement	useless (97 % disagree)	calibration mismatch (joint vs single mask)
AR-verifier teacher-forced prob	useless (mean ~ 0.003)	AR and diff have divergent distributions des
Commit-time conf, percentile rank	**works**	relative ordering picks worst placements eve

The percentile-based signal is the key insight. **All diff confidences are absolutely low** at 305 M (mean ~ 0.03), so any absolute threshold flags 100 % of positions. But the *relative ordering* encodes information about which placements are worst within a specific generation; flagging the bottom-$K\%$ produces a sensible refill rate that AR can productively fix.

Results on F10 final. We first piloted at $N = 50$ and observed `pct50` mean NLL 11.66 (apparent lift -0.90). At $N = 200$ the estimate tightens to 11.98, regressing toward the mean. We report the $N = 200$ numbers as the locked headline; the $N = 50$ pilot is left as a caveated footnote because the small-sample run informed the design.

Config	Refill %	Mean NLL	Wall (s)	Loop
Single-round K.2, $N = 200$				
`pct10`	9.4 %	12.10	0.83	7.5 %
`pct25`	25 %	12.19	0.92	12.5 %
`pct50`	**50** %	**11.98**	**1.08**	**17.5** %
`pct75`	75 %	12.37	1.22	19.0 %
Two-round K.2 (`n_outer`=2), $N = 100$				
`pct10`	18.8 %	12.04	1.06	8.0 %
`pct25`	50 %	12.24	1.23	10.0 %
`pct50`	**100** %	**11.86**	**1.55**	12.0 %
`pct75`	150 %	12.32	1.84	14.0 %
Phase J.2 baselines (same F10, same prompts, $N = 50$)				
`ar_128` (pure AR)	—	12.56	~ 6.4	—
`ms_96_32`	—	11.82	—	—
`ms_64_32`	—	11.35	—	—

Headline. At the same generation length (128 tokens), single-round `pct50` beats pure AR by -0.58 NLL/token AND is $1.3\times$ faster per generation (conservative ratio across hardware; on a single A100 with prefilled KV-cache, K.2 measures $\sim 6\times$ faster, but this depends on AR-decoder optimisation choices not controlled in our pipeline). The

$N = 50$ pilot's apparent -0.90 lift shrinks to -0.58 at $N = 200$; the gap survives, loop-rate is below baseline, and the wall-clock advantage holds. A second routing round (`n_outer`=2) yields a further marginal -0.12 NLL at the cost of 45% extra wall-clock — diminishing returns. The recipe does not beat `ms_64_32` (11.35) per-token, but `ms_64_32` generates only 64 tokens, not 128; the two configurations target different output lengths and are not directly comparable.

What this proves. The user's intuition — *diff fills the easy structure in parallel, AR fixes the suspect tokens* — holds at 305 M scale, provided the routing signal is percentile-ranked commit-time confidence rather than an absolute threshold. The recipe yields a Pareto improvement over pure AR (better NLL *and* faster wall-clock) that no single-head architecture can replicate. Iterating (`n_outer` = 2) produces a small additional gain, but the bulk of the benefit comes from the first round; we report single-round as the recommended configuration.

9.1 K.2.1 — Fine-tuning and undertrain controls

The Phase K result on F10 final raises two follow-up questions: (a) is the -0.58 NLL/token gain at `pct50` an artifact of the specific training trajectory F10 happened to follow, or does *any* composite with a sharp enough diff distribution show it? (b) is the diff distribution sharper because the model is well-trained or because it is *under*-trained (with conf collapsed on the mode of the data)? We answer both with cheap controls.

Undertrain control: F10@step 35k. We rescore K.2 on the F10 intermediate checkpoint at step 35,000 (roughly 19 % of F10's final training budget). Same architecture, same data mix, same α-schedule, but only one-fifth the gradient updates.

Config (F10 @ 35k, $N = 50$)	Mean NLL	Loop
`pct10`	12.08	0 %
`pct25`	11.97	0 %
`pct50`	**11.84**	2 %
`pct75`	12.20	12 %

F10@35k's `pct50` NLL is 11.84, essentially the same as F10 final's 11.98 at $N = 200$. **Undertraining alone does not materially shift K.2 mean NLL.** Whatever the diff-distribution property K.2 routing exploits, it stabilises early and stays roughly constant as training progresses.

Fine-tune: F11 with α fixed at 0.30. The α-schedule used in F10 is linear $1.0 \to 0.5$ over training; on average the diff head sees only $\sim 25\,\%$ of steps. We hypothesise that increasing the diff-step fraction sharpens the commit-time confidence distribution, which is exactly the signal K.2 ranks. To test, we fine-tune F10 final for 30,000 additional steps with α *fixed* at 0.30 (so 70 % of steps are diff steps, vs F10's average $\sim 25\,\%$) on the same 95 % FineWeb-Edu + 5 % Q/A mixed token stream. We call the resulting checkpoint **F11**. Cost: ~\$2 on a RTX 4090 24 GB at BS=8 (~1.8 h wall).

Head-to-head: F10 vs F11.

Config	Mean NLL F10 final	Mean NLL F11 (α=0.30)	Loop rate F10 final	Loop rate F11
`pct10`	12.10	**11.58**	7.5 %	0 %
`pct25`	12.19	**11.48**	12.5 %	0 %
`pct50`	11.98	**11.40**	17.5 %	**4** %
`pct75`	12.37	**11.72**	19.0 %	9 %

F11 beats F10 at every percentile by −0.5 to −0.7 NLL/token, with loop rate dropping by an order of magnitude (`pct50`: 17.5 % → 4 %). The α-schedule effect is real and substantial.

F11 also lifts AR-axis accuracy. On GSM8K-dev free-run accuracy ($N = 50$, decoder defaults T=0.8, top-p=0.9, rep-pen=1.15):

mode	F10 final	F11
`ar_only`	4 %	**6** %
`mode_switch_96_32`	6 %*	0 %
`mode_switch_64_32`	4 %	2 %
`paired_64_64`	0 %	4 %

*F10 number from Phase J.2; the others re-measured on a common decoder.

The AR-only lift (+2 pp) is at the edge of the $N = 50$ binomial noise floor ($\sigma \sim 2$ pp), but consistent with the NLL signal: the diff-heavy fine-tune did not damage AR-axis capability. Combined with the K.2 NLL improvement this overturns the strict reading of Phase I ("inference-time routing does not lift accuracy"): the limit was the F10 substrate, not the routing recipe.

α-sensitivity sweep (multi-seed). F11 used $\alpha = 0.30$ fixed. To map the sensitivity we trained 3 seeds per $\alpha \in \{0.20, 0.30, 0.40\}$ from F10, plus an extended F11 (F11+) that continues F11 for another 30,000 steps at $\alpha = 0.30$ fixed (single-seed). All K.2 sweeps at $N = 100$:

config	α	K.2 pct50 NLL (mean ± SD)	seeds	ar_only acc
F10 final	1.0 → 0.5	11.98 (single-seed)	1	4 %
F12-α40	0.40 fixed	11.607 ± 0.072	3	2 %
F11	**0.30 fixed**	**11.447 ± 0.038**	3	**6** %
F11+ extended	0.30 +30k more	11.26 (single-seed)	1	2 %
F12-α20	0.20 fixed	11.366 ± 0.107	3	0 %

The multi-seed (n=3) per α tightens the picture substantially. $\alpha = 0.30$ has the *tightest seed-to-seed variance* (±0.038 NLL/token, ∼ 3× smaller than $\alpha = 0.20$'s ±0.107). The apparent single-seed advantage of $\alpha = 0.20$ (−0.157 NLL vs $\alpha = 0.30$) shrinks to −0.081 in the 3-seed mean — within 1 SD of $\alpha = 0.20$'s spread. $\alpha = 0.30$ **is the Pareto-optimal recipe**: lowest mean NLL within 1 SD of $\alpha = 0.20$, best AR-axis accuracy (+2 pp over F10), *and* the most reproducible across seeds.

9.2 K.2.2 — Head-to-head against STaRR-style intra-diff routing

Concurrent work [17] (STaRR, 2601.04205) uses similar percentile-rank dynamics but routes *within* the diff head (re-mask + re-diff). We test directly whether K.2's cross-mode contribution (diff → AR refill) is the actual lever or whether the percentile-rank trick alone suffices intra-diff.

We implement a STaRR-style *intra-diff* baseline: same prefix, same diff fill, same percentile-rank routing signal — but the refill of bottom-$K\%$ positions is done by re-masking and running another diff pass (instead of AR refill). Head-to-head on the same checkpoint, same $N = 100$ prompts, same anti-rep decoder defaults:

	Mean NLL on gold (lower better)		
	STaRR-style intra-diff	K.2 cross-mode (ours)	Δ
F10 final	12.18	11.98	-0.20
F11 ($\alpha = 0.30$)	11.56	11.40	-0.16

K.2 cross-mode beats the matched intra-diff baseline by -0.16 to -0.20 NLL/token on both checkpoints. The cross-mode contribution is the actual mechanism — routing the worst-confidence positions through the AR head (which has stronger absolute calibration than the diff head at our scale) is what produces the NLL gain. The percentile-rank trick alone, applied intra-diff, gives a smaller benefit.

What this proves. K.2 is sensitive to the training α-schedule of the underlying composite. A diff-heavier fine-tune sharpens the commit-time confidence distribution and produces a Pareto improvement over the F10 baseline on *every* K.2 measurement: lower NLL, lower loop rate, equal-or-better free-run accuracy, all at the cost of a $2 fine-tune. Coupled with the F10@35k control, the explanation is specifically the α-schedule — not undertraining, not random seed, not data.

9.3 K.2.3 — Non-math substrate (WikiText-2)

A natural follow-up: is K.2's percentile-rank optimum a property of the GSM8K Q/A template, or of the composite's diff distribution itself? We rescore K.2 on 50 held-out WikiText-2-raw test passages (prefix 64 tokens of prose, gold continuation 128 tokens), F10 final, $N = 50$:

threshold	WikiText NLL	GSM8K NLL	loop rate
`pct10`	11.14	12.10	6 %
`pct25`	10.83	12.19	12 %
`pct50`	**10.38**	11.98	12 %
`pct75`	10.72	12.37	18 %

Absolute NLLs are lower on prose than on math reasoning (prose is intrinsically lower-surprise next-token prediction), so we do not compare absolute values across substrates. The relevant comparison is *shape*: `pct50` is the NLL optimum on both substrates; loop-rate ordering is preserved (`pct10` < `pct50` < `pct75`). K.2's mechanism is therefore a property of the composite's diff distribution, not an artefact of the GSM8K Q/A template.

9.4 K.2.4 — Iteration sweep: $N_{\text{outer}} \in \{1, 2, 3\}$

To map the saturation curve of cross-mode iteration we ran the three N_{outer} values back-to-back at matched $N = 100$ on F10 final (holding all other knobs constant):

threshold	N_{outer}=1	N_{outer}=2	N_{outer}=3	best at
`pct10`	12.004	12.042	12.030	—
`pct25`	11.994	12.238	**11.855**	N=3
`pct50`	12.063	**11.864**	12.153	N=2
`pct75`	12.285	12.321	12.395	—
Per-N_{outer} best config				
best NLL	11.994	**11.864**	11.855	—
best loop rate	10 %	12 %	**6** %	—

Three observations.

(i) NLL saturates after two outer iterations. Going from N_{outer}=2 best (11.864) to N_{outer}=3 best (11.855) is -0.009 NLL — within the binomial-on-NLL noise at $N = 100$. The bulk of the cross-mode iteration gain comes from the second outer round.

(ii) Loop rate continues to improve. The best-config loop rate drops 10 % → 12 % → **6** % across N_{outer}=1, 2, 3. For deployment scenarios where token-level repetition hurts more than the last 0.01 NLL, N_{outer}=3 at `pct25` is the recommended configuration.

(iii) Sweet-spot threshold shifts left with more iteration. N_{outer}=1 best is `pct25` (single-round, 25 % refill). N_{outer}=2 best is `pct50` (100 % cumulative refill across two rounds). N_{outer}=3 best is `pct25` again (75 % cumulative refill across three rounds). The optimal *total* amount of AR rewriting sits in the band 75–100 % of the diff-drafted region, and within that budget *distributing the refill across more outer rounds is strictly better than concentrating it in fewer.* Per-round refill should be more conservative as N_{outer} grows; the second-round `refill_history` arrays confirm new flag positions are not a subset of the first-round set, so each round genuinely re-evaluates which positions remain weakest.

Recommended configuration. `pct50` at N_{outer}=2 remains the default if a single number must be reported; N_{outer}=3 at `pct25` exchanges the last 0.009 NLL for a 6 pp loop-rate reduction and should be preferred whenever generation-quality matters as much as per-token NLL.

What this does not prove. Phase K targets per-token NLL on gold continuation plus wall-clock, not full GSM8K-dev free-run accuracy at parity with frontier models. The -0.58 NLL/token gain at F10 and additional -0.58 at F11 are real but small in absolute terms (we are still at ~ 11.4 NLL/token on a 50k-vocab GSM8K continuation, which is far from a frontier model's $\sim 1-2$ NLL on the same prefix). Whether the recipe transfers to (a) larger scales (≥ 1 B parameters) and (b) different α values beyond 0.30 remains open.

10 Mechanistic Interpretability — AR and diff use disjoint feature bases

A natural follow-up to the empirical claims in Sections 3–9: *do the AR head and the diffusion head use the same internal features inside the shared backbone, or do they specialise?* Phase P answers this with a mechanistic-interpretability layer built on top of the F10 (305 M, d=1024, L=20) checkpoint — no retrain, no architecture change, $\sim$ \$3 of compute. We adapt the SAE-for-diffusion methodology of DLM-Scope [19] (which studies *pure* diffusion LMs) to the composite setting, where the new question is whether the two heads sharing one backbone specialise or share features.

10.1 Setup

We adopt nnsight [6] to wrap the custom `CompositeLM` (Sfumato is not a HuggingFace model). We train TopK sparse autoencoders [7] on F10's residual stream:

- Ten per-layer SAEs on the backbone, layers 6–15 (the divergence zone identified in Phase P.0 below), AR-mode activations.
- Two pre-head SAEs on `ln_f` output, one trained on AR-mode activations and one on diff-mode activations. These let us directly compare what each head "sees".

All SAEs are k=64 TopK with 16,384 features (expansion 16× over d=1024), trained for 8,000 steps on 200 M FineWeb-Edu tokens on a single A40 (∼3 h, ∼ \$2). All checkpoints meet the standard reconstruction target (delta-loss < 0.10 on held-out tokens).

10.2 P.0 — raw activation divergence by layer

Before training any SAEs we compute, for a single GSM8K prompt run through F10 in both modes, the cosine similarity of the residual stream at each layer:

layer	cos(AR, diff)	reading
0	0.987	tok+pos emb, modes identical
5	0.880	early shared processing
10	0.650	divergence zone
14	0.510	maximum specialisation
19	0.543	slight rebound at `ln_f`

The drop is concentrated in layers 6–13. We focus SAE training there.

10.3 P.2 — cross-head SAE feature overlap (headline)

We train two TopK SAEs at the same residual position (post-`ln_f`), one on AR-mode activations and one on diff-mode activations, then compute the cosine between every AR decoder column and every diff decoder column:

metric	value
mean best-cosine (AR→diff)	0.169
median	0.129
maximum	0.904
% AR features with diff sibling ≥ 0.5	1.84 %
% AR features with diff sibling ≥ 0.7	0.42 %
% AR features with diff sibling ≥ 0.9	0.01 % (2 of 16,384)

Per the pre-committed brief hypotheses (> 0.8 = shared rep, 0.4–0.8 = partial sharing, < 0.4 = modes specialise), we land cleanly in the "modes specialise" regime. Reconciling with the raw 0.54 cosine at the final layer (P.0): that number is on dense 1024-dim activation vectors dominated by shared embedding contributions; the SAE feature-direction view at 0.169 is the decomposed, interpretable structure.

The composite at 305 M does not converge to "one representation, two readouts". The two heads operate on disjoint sparse feature bases inside the same backbone weights.

10.4 P.2b — specialisation by depth

The P.2 measurement above is at `ln_f` only (the pre-head residual). To test whether specialisation is a pre-head artefact (arising from the explicit `head_diff_proj` projection) or a backbone-wide property, we train symmetric AR-mode and diff-mode SAEs at every layer in the divergence zone (blocks 6–15, ten blocks each side) and compute cross-head best-cosine at each depth.

layer	mean cos	median	max	% sibling ≥ 0.7
block_6	0.162	0.128	0.924	0.2 %
block_7	0.159	0.128	0.905	0.1 %
block_8	0.161	0.128	0.914	0.1 %
block_9	0.162	0.127	0.924	0.1 %
block_10	0.161	0.127	0.932	0.1 %
block_11	0.159	0.126	0.913	0.1 %
block_12	0.159	0.126	0.951	0.1 %
block_13	0.158	0.126	0.845	0.1 %
block_14	0.158	0.127	0.845	0.1 %
block_15	0.157	0.127	0.904	0.2 %
`ln_f`	0.169	0.129	0.904	0.4 %

Specialisation is uniform-by-depth. Block-level mean cosine sits in the tight band $[0.157, 0.162]$ across blocks 6–15 (range 0.005, coefficient of variation $\sim 1\,\%$). The "modes specialise" finding is a backbone-wide property, not a pre-head phenomenon. Notably, `ln_f` (0.169) is slightly *less* specialised than internal blocks — opposite to what we'd expect if the explicit `head_diff_proj` layer caused the disjointness. The bridge-feature tail (cosine ≥ 0.7) accounts for under 0.5 % of features at every depth. The original P.2 measurement at `ln_f` was, if anything, a slight under-statement of how disjoint the circuits are internally.

10.5 P.3 — per-prompt confirmation across 12 prompts

We rerun on 4 GSM8K, 4 prose, and 4 explicit math-mode prompts; for each prompt we extract the top-8 firing AR features and top-8 firing diff features and check the index overlap.

Across all 12 prompts the overlap is exactly 0. The two modes fire *completely different* feature indices when processing the same prompt. Mean token-level cosine ranges from 0.439 (M1 simple arithmetic) to 0.817 (M2 chained x definition).

The 20 "bridge features" (P.2 top-20 matched pairs with cosine 0.85–0.90) appear in 1–2 top-8 slots per prompt and fire on broadly content-anchor tokens. These are candidate mode-switch routing features for the Phase K recipe.

10.6 P.4 — causal validation via steering

We pick a top AR-mode-specific feature (#9304, P.3-identified) and add $+k \times$ its SAE decoder column to F10's `ln_f` output during greedy generation. Compared to unsteered baseline on the same prompt:

- Prompt where the feature naturally fires (Janet ducks): 100 % token-equal generation at $k = 4$ and $k = 8$.

- Prompt where it does not (Roman Empire history): generation diverges by 43–64 %, breaking the AR-mode repetition loop the baseline exhibits.
- Math-chain prompt: 48 %–77 % divergence depending on mode and k.

The SAE-extracted feature directions are not just correlational — they have measurable causal influence on F10 generation. The content-conditional behaviour (large effect only on prompts where the feature already participates) is the expected pattern for a learned direction.

10.7 What this opens

Distillation candidate. If AR and diff feature bases are near-disjoint, F10's backbone is effectively two specialised subnetworks sharing only the bottom-most layers and a small "bridge" subspace. A distillation into AR-only and diff-only models with shared embeddings could recover most of F10's capability at ∼half the parameter count. Worth a paper-D scaling experiment.

Phase K bridge features. The ∼20 high-cosine matched pairs from P.2 are the natural candidates for the "mode-switch routing" features that K.2's per-token cross-mode recipe (§9) implicitly uses. They fire on universal content tokens across all prompt types tested. Future work: confirm they are the actual mechanism via attribution graphs.

Honest negative. The interpretability story does not rescue F10's free-run accuracy (still ∼6 % on GSM8K-dev). Mechanistic insight here is about *what the model is computing*, not about closing the capability gap. We document this so readers do not over-claim the interpretability layer.

Total interpretability cost. ∼ $3 on one A40 for ∼3 h (12 SAEs + overlap analysis). Reproducible via the recipe at `https://github.com/eren23/sfumato/tree/main/e5/interp` and the SAE weights at `https://huggingface.co/eren23/sfumato-composite-ckpts/tree/main/interp/saes`.

11 Discussion

11.1 When to use composite training

The trade-off has a clean structure:

- **Strong yes** when (a) data is constrained ($\lesssim$ 1,000 problems on a math substrate), (b) the deployment scenario can exploit mode-switching inference, and (c) the diffusion-axis performance matters.
- **Mild yes** when (a) data is abundant but the diffusion-axis advantage outweighs the small AR tax for the application, and (b) OOD prose generalisation is not a priority.
- **No** when (a) the deployment is pure-AR (no mode-switching), (b) the only metric is AR-mode held-out perplexity, or (c) OOD prose generalisation matters.

11.2 Why the crossover happens

We do not provide a formal account of *why* the crossover sits near $\sim 1{,}000$ problems on GSM8K. Two non-exclusive hypotheses:

1. *Regularisation budget.* The diffusion-mode mask-fill objective acts as an implicit regulariser on the shared backbone. In the data-scarce regime, the AR-only model overfits to specific token sequences in GSM8K-train, while composite generalises better via the denoising objective. As data grows, the AR-only baseline gets the regularisation it needs from the data itself, and composite's diffusion objective becomes a tax rather than an aid.

2. *Capacity allocation.* The shared backbone has finite capacity. With scarce AR signal, it cannot afford to specialise in next-token prediction and the bidirectional/diffusion component provides complementary signal. With abundant AR signal, specialisation in AR pays more than the diffusion signal costs.

11.3 Relationship to published work

DiFFPO [20] reports discrete-diffusion outperforming AR in data-constrained settings via RL fine-tuning. Our result is a *from-scratch joint-training* parallel: same regime (data scarcity), same direction (diffusion-flavoured training wins), no RL. This is the first evidence we are aware of that the DiFFPO regime effect manifests in pre-training as well, at small scale and without RL.

The compute-matched control closes a gap in BD3-LMs [12] and Transfusion [28], neither of which explicitly compares against a compute-matched pure-diffusion baseline. Our result is small-scale (60M–300M), but the apples-to-apples comparison is cleaner than in works that focus on absolute scale.

11.4 What this study does not claim

- That composite training is universally better than separated training. The trade depends on data regime and which axis matters.

- That the crossover at $\sim$ 1k problems generalises beyond the GSM8K substrate. A different substrate would produce a different curve.

- That mode-switching reliably wins downstream tasks. We have $n = 3$ data with high per-seed variance; the $n = 8$ expansion tightens this but does not eliminate it.

- That toy-scale results scale to 1B+ parameters. The trade direction may shift with model scale; we cannot test.

12 Related Work

12.1 Joint AR + diffusion training

BD3-LMs [12] train a block-AR + within-block discrete-diffusion model on the OpenWebText corpus at $\sim$ 400M parameters and find competitive perplexity vs. standard AR. They do not report a parameter-matched pure-AR baseline trained on the same data, nor a compute-matched control on the diffusion side.

Transfusion [28] trains a 7B single-transformer model with both an AR head (for text) and a continuous-diffusion head (for images). The work demonstrates joint training is feasible at scale and beats quantised-image-AR baselines. The training is not designed for parameter-matched comparison against pure-AR text models.

Janus and **Janus-Pro** [2, 21] train unified 7B models with separate visual-understanding and visual-generation encoders, with AR text generation. The architecture is closer to multi-modal AR than to AR+diffusion-text training.

MonoFormer [27] is the closest architectural precedent: one transformer serves both AR (text) and diffusion (images), switching between causal and bidirectional attention masks — exactly the mechanism we use. It is cross-modal (text vs. image), demonstrates joint training is feasible, and does not analyse whether the two modes share or specialise internal features. We are text-only and add the mechanistic analysis (§10).

Dual-objective Language Models [15] are the nearest *text-only* neighbour: they combine AR and masked-diffusion objectives on a single backbone with no architectural modification, train 50 models across data-repetition settings, and conclude it is "optimal to combine both objectives under all evaluated settings." Our data-efficiency curve refines this: we find a *crossover* — composite wins the AR axis below $\sim 1{,}000$ math problems but pays a measurable tax above it (§3) — so "always combine" holds for the diffusion axis and the low-data AR axis, but not for the AR axis in the high-data regime. We also add the compute-matched control, the SAE specialisation analysis, and the Phase K inference recipe, none of which appear in that work.

12.2 Pre-trained AR $\rightarrow$ diffusion adaptation

DiffuLLaMA [9] adapts a pre-trained AR LLaMA backbone to diffusion mode via continued training. The reported result is "competitive, not superior" vs. AR baselines. This is closely related to our compute-matched control but starts from a different initialisation; we train from scratch.

Fast-dLLM v2 [23] adapts a pre-trained AR model to block-diffusion mode and reports a 2.5× inference speedup at $<$ 1B tokens.

12.3 Continuous flow language models

ELF [1] demonstrates pure continuous flow language modelling. We replicated a small-scale flow variant (`ARFlowLM`) as part of our T1 ablations and found 0% accuracy at 60M-3k on GSM8K-dev — flow-only models do not compose with AR at our toy scale. We do not extend this branch.

12.4 Data-constrained discrete diffusion

DiFFPO [20] shows discrete diffusion outperforms AR in data-constrained settings via RL fine-tuning. Our data-efficiency curve replicates this regime effect in *from-scratch joint training* (no RL), at 200M scale, on math-reasoning rather than the language-modelling substrate DiFFPO used.

Super data learners [8] attribute the data efficiency of diffusion LMs specifically to *random masking acting as stochastic regularisation* (with comparable gains from MLP dropout and weight decay), not to the diffusion objective per se. This is independent support for the regularisation reading of our crossover (§3 discussion): in the data-scarce regime the diffusion head's masking regularises the shared backbone, and the benefit fades once the AR signal is abundant enough to no longer need it.

12.5 Mode-switching and interleaved inference

The original Sfumato planning document proposed a learned mode-router that selects among AR-extend, diffuse-current-block, re-diffuse-last-K-blocks at every sub-block boundary. We do not train such a router in this work; instead we test fixed mode-switching policies (e.g., AR-then-diffusion-revise) as the inference recipe. **SongBloom** [18] interleaves AR-sketch and diffusion-refine for music; the loop pattern is analogous but the domain is different.

12.6 Mechanistic interpretability of diffusion LMs

DLM-Scope [19] is, to our knowledge, the first SAE-based interpretability framework for diffusion language models; it trains sparse autoencoders on *pure* discrete-diffusion models (e.g. Dream, LLaDA) and shows the features support diffusion-time interventions and decoding-order selection. Our Phase P analysis (§10) extends the SAE methodology to a *composite two-head* model and asks a question that does not arise in the pure-diffusion setting: *do the AR head and the diffusion head, sharing one backbone, converge to the same features or specialise into different ones?* We find near-disjoint sparse feature bases (mean cross-head cosine 0.157–0.169, uniform across all ten backbone blocks), which to our knowledge is not reported for any shared-backbone AR+diffusion model. DLM-Scope provides the methodological foundation; the composite mode-specialisation result is new.

13 Limitations and Future Work

13.1 Scale

All experiments are at 60M–300M parameters. The trade-off characterisation may shift at $\geq$ 1B; published works at 7B [9, 28] report different direction signals. We treat the 60M–300M characterisation as "small-scale evidence" and explicitly do not extrapolate.

13.2 Substrate

All training is on GSM8K-train ($\sim$ 4M tokens, math-reasoning domain). The data-efficiency crossover is substrate-specific; we make no claim that it generalises to broader corpora.

13.3 Eval metric

Downstream accuracy on GSM8K-dev is binomial-noise dominated at this scale (<6% correct out of 50 in most cells). We rely on continuous NLL metrics throughout, but downstream-task accuracy is the metric practitioners care about. Mode-switching at $n = 8$ tightens the CI but does not eliminate the noise.

13.4 Single hyperparameter trace

The α-schedule (linear $1.0 \rightarrow 0.5$), mask-ratio distribution (Uniform$(0.1, 0.9)$), and learning rate (6×10^{-4} with cosine decay) were picked from BD3-LMs defaults and not swept. A small α-schedule sweep is the natural follow-up.

13.5 Routing scope (Phase I narrow null)

Phase I (Section 8) tested both scripted multi-round AR$\leftrightarrow$diff interleaving and three non-learned heuristic routers on F9 (FineWeb-only) and F10 across multiple training stages. On F9 the diff-head loop pathology let fine-grained iteration look like a win (+2 pp accuracy, −22 pp loop rate over the single-switch baseline). On F10 at convergence the same iteration ties pure AR (4 % accuracy), and no heuristic crosses the +4 pp gate over the best fixed schedule. Per the Phase I decision rule we did not launch a learned router. The honest scope-limit is specific: *routing-during-AR-generation for accuracy lift* helps when the diff head is broken, and disappears when proper training (F10's Q/A mix) fixes the diff head directly. A learned router may still earn its weight at scales where AR and diff heads diverge more under joint training (Phase I.2/I.3 future work).

This null sits alongside three positive Phase J measurements (Section 8.5): the composite still delivers a 5.43× wall-clock speedup via parallel diff-fill, a FIM capability AR structurally lacks, and a −0.74 NLL/token revision improvement on gold continuation. The Phase I null and the Phase J wins are independent claims about different axes; the null does not undermine the wins.

13.6 Future work

1. Test the data-efficiency crossover on a non-math substrate (e.g., a $\sim$ 10M-token slice of a code corpus).
2. Extend the compute-matched control to AR-only-at-2× as well, to test "does composite also beat compute-matched pure-AR on its own axis at small data?"
3. Rerun Phase I (interleaving + heuristic routers) on F10 at convergence (step 183k, not yet available at submission). If the routing-vs-fixed picture shifts as the diff head specialises further, a REINFORCE-trained router becomes worth attempting; otherwise the negative reported here is the final word at $\sim$300 M scale.
4. Replicate at 1B from scratch to test whether the crossover *and* the routing-versus-fixed picture shift with scale.

References

[1] others Chen. Elf: Continuous flow language models. 2026. arXiv:2605.10938.

[2] Xiaokang Chen et al. Janus-pro: Unified multimodal understanding and generation with data and model scaling. 2025. arXiv:2501.17811.

[3] Zicong Cheng et al. Deer: Draft with diffusion, verify with autoregressive models. 2025. arXiv:2512.15176.

[4] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. In *NeurIPS Datasets and Benchmarks*, 2021.

[5] Xun Fang, Yunchen Li, Hang Yuan, and Zhou Yu. Factorization-error-free discrete diffusion language model via speculative decoding. 2026. arXiv:2605.14305.

[6] Jaden Fiotto-Kaufman et al. Nnsight and ndif: Democratizing access to open-weight foundation model internals. 2024. arXiv:2407.14561.

[7] Leo Gao, Tom Dupré la Tour, Henk Tillman, Gabriel Goh, Rajan Troll, Alec Radford, Ilya Sutskever, Jan Leike, and Jeffrey Wu. Scaling and evaluating sparse autoencoders. 2024. arXiv:2406.04093.

[8] Zitian Gao, Haoming Luo, Lynx Chen, Jason Klein Liu, Ran Tao, Joey Zhou, and Bryan Dai. What makes diffusion language models super data learners? 2025. arXiv:2510.04071.

[9] Shansan Gong et al. Diffullama: Scaling ar diffusion language models. In *ICLR*, 2025. arXiv:2410.17891.

[10] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. In *ICLR*, 2020.

[11] Guanghao Li et al. Diffuspec: Unlocking diffusion language models for speculative decoding. 2025. arXiv:2510.02358.

[12] Shen Nie et al. Block discrete denoising diffusion language models. In *ICLR*, 2025. arXiv:2503.09573.

[13] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI Blog*, 2019.

[14] Subham Sekhar Sahoo et al. Simple and effective masked diffusion language models. In *NeurIPS*, 2024.

[15] David Samuel and Lucas Georges Gabriel Charpentier. Dual-objective language models: Training efficiency without overfitting. 2025. arXiv:2512.14549; revised 2026.

[16] Yixuan Su, Tian Lan, Yan Wang, Dani Yogatama, Lingpeng Kong, and Nigel Collier. A contrastive framework for neural text generation. In *NeurIPS*, 2022.

[17] Xinhao Sun et al. Starr: Spatial-temporal token-dynamics-aware responsive remasking for diffusion language models. 2026. arXiv:2601.04205.

[18] others Wang. Songbloom: Interleaved ar sketch + diffusion refine for music. 2025. arXiv:2506.07634.

[19] Xu Wang, Bingqing Jiang, Yu Wan, Baosong Yang, Lingpeng Kong, and Difan Zou. Dlm-scope: Mechanistic interpretability of diffusion language models via sparse autoencoders. 2026. arXiv:2602.05859.

[20] others Wei. Diffpo: Discrete-diffusion flow preference optimization. 2025. arXiv:2510.02212.

[21] Chengyue Wu et al. Janus: Decoupling visual encoding for unified multimodal understanding and generation. 2024. arXiv:2410.13848.

[22] Chengyue Wu et al. Fast-dllm v2: Efficient block-diffusion llm. 2025. arXiv:2509.26328.

[23] others Wu. Fast-dllm v2: Block diffusion adaptation of pretrained llms. 2025. arXiv:2509.26328.

[24] Yifan Yu et al. Introspective diffusion language models. 2026. arXiv:2604.11035.

[25] Shuangfei Zhai et al. Exclusive self attention. 2026. arXiv:2603.09078.

[26] Shuibai Zhang et al. Corrective diffusion language models. 2025. arXiv:2512.15596.

[27] Chuyang Zhao, Yuxing Song, Wenhao Wang, Haocheng Feng, Errui Ding, Yifan Sun, Xinyan Xiao, and Jingdong Wang. Monoformer: One transformer for both diffusion and autoregression. 2024. arXiv:2409.16280.

[28] Chunting Zhou et al. Transfusion: Predict the next token and diffuse images with one multi-modal model. 2024. arXiv:2408.11039.